4 Function və stored procedure nə ilə fərqlənir

Kafedə iki ayrı ehtiyacımız var. Birincisi, məbləğdən endirimi çıxıb cavab almaqdır. İkincisi, sifarişin vəziyyətini bazada dəyişməkdir. Birinci işin nəticəsi hesablanmış dəyərdir; ikinci iş bazada əməliyyatdır.

SQL Server-də **function** bir dəyər və ya cədvəl nəticəsini sorğunun içində istifadə etmək üçün verir. **Stored procedure** isə adla saxlanmış əmrləri icra edir. Hər ikisi bazada saxlanan təkrar istifadə olunan kod ola bilər.

SQL cədvəllərlə işləmək üçün dildir. `SELECT` məlumatı oxuyur, `INSERT` yeni sətir əlavə edir, `UPDATE` mövcud sətri dəyişir, `DELETE` sətri silir. SQL Server-də əlavə imkanları olan bu dil variantı **T-SQL** adlanır.

İki ehtiyac

```
"100-den 15 cix, cavabi ver." -> Function ucun numune.

"42-ci sifarisi tesdiqle."   -> Procedure ucun numune.
```

Bu dərsdəki function məhdudiyyətləri **SQL Server T-SQL funksiyalarına** aiddir. PostgreSQL kimi başqa bazalarda eyni qaydaları yoxlamadan tətbiq etmək olmaz.

Scalar function bir dəyər qaytarır

Endirimdən sonrakı məbləği hesablayaq:

T-SQL

```
CREATE OR ALTER FUNCTION dbo.NetAmount
(
    @Gross decimal(12,2),
    @Discount decimal(12,2)
)
RETURNS decimal(12,2)
AS
BEGIN
    RETURN @Gross - @Discount;
END;
GO

SELECT dbo.NetAmount(100, 15) AS Net;
```

Nəticə **85.00** olur. Tək nəticə dəyərinə **scalar** deyilir. `@Gross` və `@Discount` giriş parametrləridir: çağırış zamanı verilən dəyərlər. `decimal(12,2)` onluq tipdir; cəmi 12 rəqəm, onlardan 2-si vergüldən sonra ola bilər.

`RETURNS` nəticənin tipini, `RETURN` nəticənin özünü bildirir. `AS Net` cavab sütununa Net adını verir. dbo obyektin aid olduğu schema-dır. **Schema** baza obyektlərini ad altında qruplaşdırır.

`GO` SQL Server-i idarə etmək üçün istifadə olunan alətlərdə əmrlər qrupunu ayırır. Serverin adi T-SQL əmri deyil. Tətbiqdən göndərərkən həmin qruplar ayrı göndərilir.

Əsas fərq: function nəticəsini başqa SELECT ifadəsinin içində istifadə edə bilirik.

Procedure əmrləri adla icra edir

Orders cədvəlində `Id`, `CustomerId` və `Total` sütunlarının olduğunu qəbul edək. Müştərinin sifarişlərini gətirən procedure yazaq:

T-SQL

```
CREATE OR ALTER PROCEDURE dbo.GetOrders
    @CustomerId int
AS
BEGIN
    SET NOCOUNT ON;

    SELECT Id, Total
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId;
END;
GO

EXEC dbo.GetOrders @CustomerId = 7;
```

`EXEC` procedure-u çağırır. `int` tam ədəd tipidir. `FROM` məlumatın hansı cədvəldən alınacağını, `WHERE` isə hansı sətirlərin seçiləcəyini bildirir. Bu procedure müştəri 7-nin sifariş cədvəlini qaytarır.

Qayıdan sətir və sütunlar toplusuna **result set** deyilir. Deməli, “procedure heç nə qaytarmır” düzgün izah deyil.

`SET NOCOUNT ON` təsirlənən sətir sayı barədə əlavə mesajları azaldır. SELECT nəticəsini gizlətmir. Onu hər sorğunu böyük faizlə sürətləndirən qayda kimi əzbərləmə.

Procedure SELECT edə bildiyi kimi INSERT, UPDATE və DELETE də edə bilər. SQL Server-də istifadəçinin yazdığı T-SQL function isə bazanın xarici vəziyyətini dəyişən belə iş vahidi üçün nəzərdə tutulmur.

Procedure nəticəni üç ayrı yolla verə bilər

Əvvəlki səhifədə sifariş cədvəlini SELECT ilə aldıq. Procedure ayrıca çıxış parametrini də doldura bilər:

T-SQL

```
CREATE OR ALTER PROCEDURE dbo.CountOrders
    @CustomerId int,
    @Count int OUTPUT
AS
BEGIN
    SELECT @Count = COUNT(*)
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId;

    RETURN 0;
END;
GO

DECLARE @N int, @Status int;
EXEC @Status = dbo.CountOrders
    @CustomerId = 7, @Count = @N OUTPUT;
SELECT @N AS OrderCount, @Status AS Status;
```

`COUNT(*)` sətirləri sayır. `OUTPUT` procedure-un doldurduğu dəyəri çağırana çıxarır. `@N` say olur. `RETURN 0` isə bu nümunədə uğuru bildirən tam ədəd statusudur; `@Status` onu alır. `DECLARE` dəyişən elan edir.

Üç yolu ayır: **SELECT nəticə cədvəli verir; OUTPUT parametr dəyəri verir; procedure RETURN-u tam ədəd status verir.** Procedure RETURN-u function-un məbləğ qaytaran RETURN-u ilə eyniləşdirmə.

Table valued function cədvəl kimi istifadə olunur

Function yalnız bir rəqəm qaytarmağa məcbur deyil. **Table-valued function**, qısa **TVF**, cədvəl qaytarır. Onu başqa sorğunun məlumat mənbəyi kimi işlətmək olur.

T-SQL

```
CREATE OR ALTER FUNCTION dbo.CustomerOrders
(
    @CustomerId int
)
RETURNS TABLE
AS
RETURN
(
    SELECT Id, Total
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId
);
GO

SELECT Id, Total
FROM dbo.CustomerOrders(7)
WHERE Total > 20;
```

Function əvvəl müştərinin sifarişlərini müəyyən edir. Xarici sorğu isə nəticəni `Total > 20` ilə daraldır. Belə əlaqələndirilə bilmə function-un vacib üstünlüyüdür. Procedure normal SELECT-də bu cür birbaşa FROM mənbəyi kimi yazılmır.

Bir SELECT ilə yazılan bu formaya **inline TVF** deyilir. Baza onu əsas sorğu ilə birlikdə qiymətləndirə bilər.

Multi-statement TVF isə əvvəl nəticə cədvəlini dəyişən kimi yaradır, bir neçə əmrlə doldurur və qaytarır. Daha çox addım yazmaq mümkündür, amma planın seçilməsi çətinləşə bilər. Daha mürəkkəb forma avtomatik daha sürətli forma deyil.

Birlikdə tamamlanmalı dəyişikliklər transaction istəyir

Sifariş təsdiqlənəndə həm Orders vəziyyəti dəyişsin, həm məhsul ehtiyatı azalsın. Birinci dəyişiklik yazılıb ikinci uğursuz olsa, məlumat yarımçıq qalar.

Transaction birlikdə təsdiqlənməli baza işlərinin sərhədidir. `COMMIT` təsdiqləyir, `ROLLBACK` həmin transaction-un dəyişikliklərini geri alır. Bu birlikdəlik xüsusiyyətinə **atomiklik** deyilir.

Məntiqi axın

```
Transaction basla.
    Sifarisi tesdiqle.
    Ehtiyati azalt.

Her ikisi ugurludur -> COMMIT.
Xeta var           -> ROLLBACK.
```

Procedure transaction və xəta idarəetmə məntiqi daşıya bilər. T-SQL UDF, yəni istifadəçinin yaratdığı SQL Server function-u, bunun üçün uyğun iş vahidi deyil.

`TRY/CATCH` xətalı icranı tutmaq üçün bloklardır. `THROW` xətanı çağırana ötürür. Əsas fikir xəta gizlədib uğur demək əvəzinə nəticəni düzgün bildirməkdir.

Transaction bazadan kənara göndərilmiş email-i geri ala bilmir. Eyni qaydada başqa transaction-un içində çağırılan procedure-un daxili COMMIT-i müstəqil xarici təsdiq deyil. Transaction-un kim tərəfindən açıldığını bilmək lazımdır.

Seçim: bir-birinə bağlı baza dəyişikliklərini təşkil etmək üçün procedure və ya tətbiqin idarə etdiyi transaction düşün. Function-u nəticə ifadəsi kimi seç.

Performans və təhlükəsizlik barədə düzgün nəticə

Baza sorğunu icra etmək üçün yol seçir. Həmin yol **execution plan**, icra planıdır. Planın sonradan təkrar istifadə üçün saxlanmasına **plan cache** deyilir.

Procedure çox vaxt planı təkrar istifadə edə bilər. Amma plan yenidən qurula bilər və parametrli adi SQL də təkrar istifadədən yararlana bilər. Ona görə “procedure həmişə əvvəlcədən hazırdır və daha sürətlidir” doğru nəticə deyil.

Eyni function-u çox sətir üçün ayrı hesablatmaq da baha ola bilər. Bəzi SQL Server versiya və şərtlərində function əsas plana açıla bilər; buna **inlining** deyilir. Sürət barədə son qərar real plana və ölçməyə əsaslanır.

SQL injection istifadəçi mətninin məlumat əvəzinə SQL əmrinə çevrilməsidir. Procedure daxilində istifadəçi mətnini SQL-ə birləşdirmək bu riski yenə yarada bilər. Dəyərləri ayrıca parametr kimi ötürmək lazımdır.

Özünü yoxla

Hesablanmış məbləği SELECT sütunu kimi istəyirsən: scalar function və ya sadə ifadə düşün.

Sifariş nəticəsini başqa cədvəllə birləşdirmək istəyirsən: inline TVF uyğun ola bilər.

Sifarişi, ehtiyatı və audit qeydini dəyişmək istəyirsən: procedure və transaction sərhədini düşün. Audit baş vermiş əməliyyatın qeydidir.